

ros2_kafka_dispatcher — Node Description

Overview

`ros2_kafka_dispatcher` is a ROS 2 software system that bridges the ROS 2 topic graph to external message brokers — Apache Kafka and MQTT. It acts as the IoT data source layer of the GIS4IoT architecture: field robots running ROS 2 publish telemetry streams, and this system ingests, serializes, and forwards them to the Kafka or MQTT messaging backbone consumed by stream processing engines such as `ksqlDB`, Apache Flink, or `NebulaStream`.

The pipeline is composable and lifecycle-managed. All subscriptions are created generically using `rosidl` type introspection — no message type is hardcoded. This decouples robot-side middleware from the cloud processing tier and satisfies the GIS4IoT requirements for dynamic device deployment and intermittent-connectivity tolerance.

For the full component topology and data flow diagrams, see [Architecture](#).

Runtime Configuration

The `dispatcher_controller` node is the control plane. It supports three topic selection modes:

Mode	Description
<code>file</code>	Reads topic list from a YAML file on disk. Default.
<code>gui</code>	Waits for an <code>apply_selection</code> service call from an external client.

Mode	Description
all	Auto-discovers all active ROS 2 topics via <code>introspection_manager</code> .

In `file` mode, each entry in the selection YAML specifies the topic name, message type, and an optional `topic_tools` plugin block:

```
- topic_name: /robot/odom
  msg_type: nav_msgs/msg/Odometry

- topic_name: /camera/image_raw
  msg_type: sensor_msgs/msg/Image
  topic_tools:
    plugin: topic_tools::ThrottleNode
    output_topic: /throttled/image_raw
    parameters:
      period: 0.2    # forward at 5 Hz
```

In `all` mode, discovery is bounded by `all_mode_max_topics` and filtered with `all_mode_allowlist` / `all_mode_denylist` glob patterns. Setting `auto_apply_on_mode_change: true` (default) activates the selection immediately when the mode is switched at runtime.

Topic Mapping and Multi-Sensor Compatibility

`kafka_sink` and `mosquitto_sink` subscribe to topics using `rosidl` type introspection, which resolves the message layout at runtime from the type support libraries installed in the ROS 2 environment. This makes the system compatible with any ROS 2 message type without recompilation:

- Localisation: `nav_msgs/msg/Odometry`, `sensor_msgs/msg/NavSatFix`

- Perception: `sensor_msgs/msg/PointCloud2` ,
`sensor_msgs/msg/Image` ,
`sensor_msgs/msg/LaserScan`
- Inertial: `sensor_msgs/msg/Imu`
- Environment: `sensor_msgs/msg/Temperature` ,
`sensor_msgs/msg/RelativeHumidity`
- Custom project-specific message types

topic_tools integration

Each topic in the selection can be routed through a composable `topic_tools` plugin before reaching the sink. The plugin is loaded into a shared component container via `composition_interfaces/srv/LoadNode` . The contract is strictly one input topic → one output topic; the sink subscribes to the plugin output rather than the original topic.

Plugin	Key parameter	Effect
<code>topic_tools::ThrottleNode</code>	<code>period</code> (s)	Rate-limits output; excess messages silently dropped
<code>topic_tools::DropNode</code>	<code>x</code>	Forwards every x-th message, count-based decimation
<code>topic_tools::DelayNode</code>	<code>delay</code> (s)	Republishes after a fixed delay; useful for sensor time-alignment

Plugin	Key parameter	Effect
<code>topic_tools::RelayNode</code>	—	Topic rename or namespace bridge without data modification
<code>topic_tools::RelayFieldNode</code>	<code>expression</code> , <code>output_type</code>	Extracts a single field from the message

For details on unsupported nodes and architectural constraints, see [Architecture — topic_tools integration](#).

Kafka topic naming

The Kafka topic name is derived from the ROS 2 topic name using one of two modes, set via `kafka.topic_mapping_mode` :

- `prefix_ros_topic` (default): `/robot/odom` → `ros2.robot.odom`
- `fixed` : all messages go to a single topic defined by `kafka.fixed_topic`

A per-topic `kafka_name` field in the subscription YAML overrides the derived name for that topic.

Schema Transformation

Serialization is performed on the sink side using `rosidl` type introspection. No external schema registry is required.

Serialization formats — selected per sink via

`kafka.payload_format` or `mqtt.payload_format` :

Format	Characteristics
<code>cdr</code>	Binary CDR encoding. Compact and lossless; requires type information for deserialization. Default.
<code>json</code>	Human-readable JSON. Larger payload; directly consumable by GIS tools and REST clients.

Kafka record headers — added to every produced record:

Header	Value
<code>ros_topic</code>	Original ROS 2 topic name
<code>ros_type</code>	Message type, e.g. <code>nav_msgs/msg/Odometry</code>
<code>payload_format</code>	<code>cdr</code> or <code>json</code>
<code>stamp_ms</code>	Message timestamp in milliseconds
<code>kafka_topic</code>	Kafka topic the record was produced to

These headers allow downstream consumers to deserialize records correctly without out-of-band configuration.

CDR-to-JSON conversion: `kafka_cdr_to_json` is a lifecycle node that reads CDR-encoded Kafka records and republishes them as JSON to new Kafka topics (appending a `.json` suffix by default). This supports a CDR-first ingestion strategy: compact binary encoding on the robot-to-broker link, converted to JSON for GIS-compatible downstream consumers without re-publishing from the robot side.

Field extraction: `RelayFieldNode` with a Python-style expression (e.g. `m.pose.pose.position.x`) and a target `output_type` strips a large message down to a single scalar or sub-message before serialization. This reduces payload size on bandwidth-constrained robot-to-cloud links.

QoS-Aware Forwarding

Forwarding behavior is independently configurable at the ROS 2 subscription layer and at the broker layer.

ROS 2 subscription QoS: - `qos_depth` : history queue depth applied uniformly to all subscriptions created by `kafka_sink` and `mosquitto_sink`. Default: 10. - `introspection_manager` publisher: `publisher_reliability` (`reliable` / `best_effort`) and `publisher_durability` (`volatile` / `transient_local`) are set independently to match the QoS profile of the monitored graph.

Kafka producer: - `kafka.acks` (0 , 1 , all): acknowledgement level traded against throughput. - `kafka.max_queue_messages` + `kafka.drop_when_full` : bounded in-process send queue with a configurable drop-when-full policy for handling producer backpressure without blocking ROS 2 callbacks. - `kafka.linger_ms` / `kafka.batch_size` : batching parameters passed through to `librdkafka`.

MQTT: - `mqtt.qos` levels 0, 1, and 2 are supported. - Last Will and Testament (LWT) is configured via `mqtt.lwt_topic` and `mqtt.lwt_payload`, enabling broker-side notification when the sink disconnects unexpectedly. - TLS encryption is enabled via `mqtt.use_tls` and `mqtt.ca_cert_path`.

Dynamic Reconfiguration

The system supports full topic selection changes at runtime without restarting any node.

Control services on `dispatcher_controller`:

Service	Type	Effect
<code>apply_selection</code>	<code>ApplySelection</code>	Accepts a new topic list directly;

Service	Type	Effect
		triggers a full reconfiguration cycle.
<code>reload_selection</code>	<code>ReloadSelection</code>	Re-reads the current source: re-parses YAML in <code>file</code> mode, re-discovers in <code>all</code> mode.

On each apply cycle, `dispatcher_controller` : 1. Calls `deactivate` on all active sinks. 2. Serializes the new topic list to `subscriptions_yaml` . 3. Unloads previous `topic_tools` plugin components and loads new ones via `LoadNode` . 4. Calls `configure` → `activate` on the sinks.

The controller exposes its current phase for external polling:

Phase	Meaning
<code>IDLE</code>	Ready to accept a new selection.
<code>BUSY</code>	Reconfiguration in progress.
<code>ERROR</code>	Last operation failed; details in <code>last_error</code> .

This mechanism directly addresses the GIS4IoT dynamic deployment requirement: when an IoT device connects or reconnects to the network, its topics can be added to the active selection and forwarded to the Kafka backbone without system restart or manual node lifecycle management.

For full service definitions, see [API Reference — services](#).

Component Summary

Component	Type	Role
<code>introspection_manager</code>	Node	Monitors the ROS 2 graph; publishes discovered topic names and types
<code>dispatcher_controller</code>	Node	Control plane: selection modes, sink lifecycle orchestration, plugin management
<code>kafka_sink</code>	Lifecycle Node	Subscribes to selected topics; serializes and produces records to Apache Kafka
<code>mosquitto_sink</code>	Lifecycle Node	Subscribes to selected topics; forwards messages to an MQTT broker
<code>kafka_source</code>	Lifecycle Node	Consumes Kafka topics and republishes messages as ROS 2 topics
<code>kafka_cdr_to_json</code>	Lifecycle Node	Reads CDR-encoded Kafka records; republishes as JSON to new topics
<code>kafka_client</code>	Library	<code>librdkafka</code> wrapper: async producer queue, delivery callbacks, Kafka header support